LeetCode Practice

7 Easy: Binary Search

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

7
EASY

Patterns (in order)

1. Binary Search (7)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

7 questions | tap a title to jump to it

Easy questions (7) Go >

Binary Search (7) Go >

E 1. Search Insert Position LC 35

E 2. Sqrt(x) LC 69

E 3. First Bad Version LC 278

E 4. Valid Perfect Square LC 367

E 5. Guess Number Higher or Lower LC 374

E 6. Binary Search LC 704

E 7. Find Smallest Letter Greater Than Target LC 744

Easy questions (7)

EASY | TOPIC 1 OF 1

Binary Search

7 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Search Insert Position LC 35
2. Sqrt(x) LC 69
3. First Bad Version LC 278
4. Valid Perfect Square LC 367
5. Guess Number Higher or Lower LC 374
6. Binary Search LC 704
7. Find Smallest Letter Greater Than Target LC 744

EASY | Binary Search | LeetCode #35 | 1 of 7 | 1/7

Search Insert Position

QUESTION

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with O(log n) runtime complexity.

Example 1	
Input	`nums = [1,3,5,6], target = 5`
Output	`2`

Example 2	
Input	`nums = [1,3,5,6], target = 2`
Output	`1`

Example 3	
Input	`nums = [1,3,5,6], target = 7`
Output	`4`

CONSTRAINTS

- 1 <= nums.length <= 10^4
- 10^4 <= nums[i] <= 10^4
- nums contains distinct values sorted in ascending order.
- 10^4 <= target <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Sorted array of distinct integers. Return the index of target if found,

otherwise return the index where it would be inserted to keep order. Right?

Must run in O(log n) — so binary search is required."

"[1,3,5,6], target=5 -> index 2 (found) ok

[1,3,5,6], target=2 -> index 1 (between 1 and 3) ok

[1,3,5,6], target=7 -> index 4 (after all) ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    if nums[i] >= target:
        return i
return n
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Linear scan of a sorted array. Binary search for the first element >= target is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Standard binary search returning lo — O(log n). Walk-through:"

lo=0, hi=len(nums). While lo < hi:

`mid = (lo+hi)//2.`

If nums[mid] < target: lo = mid+1.

Else: hi = mid.

Return lo.

WHY hi = len(nums): if target > all elements, lo lands at len(nums) = insert at end.

WHY return lo: after the loop, lo == hi = the first position where nums[i] >= target.

This is exactly the insert position: target would go here to keep array sorted.

WHY lo < hi (not <=): when lo==hi the search space is empty -> that's the answer.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _35_SearchInsertPosition:
    def searchInsert(self, nums: List[int], target: int) -
> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            elif target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        return left
```

PHASE 5 TEST & DEBUG

"Let me trace searchInsert([1,3,5,6], target=5)."

lo=0, hi=3. mid=1: nums[1]=3<5 -> lo=2.

lo=2, hi=3. mid=2: nums[2]=5==5 -> return 2 ok

target=2: lo=0,hi=3. mid=1: 3>2->hi=0. lo=0,hi=0: mid=0: 1<2->lo=1. lo=1>hi=0 -> return lo=1 ok

target=7: lo always moves right. Eventually lo=4>hi=3 -> return 4 ok (insert at end).

WHY return lo: when loop ends, lo is the insertion point (leftmost position >= target).

[1,3,5,6], target=7: all elements < 7 -> lo keeps moving right -> lo=4=n -> insert at end ok

PHASE 5 TEST & DEBUG

WHY this is exactly bisect_left: Python's bisect.bisect_left(nums,target) does same thing.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

The 'return left' at the end is the core insight — left always converges

to the leftmost position where target can fit. This same pattern appears

in lower_bound in C++ and bisect_left in Python.

Given more time I'd ask: what if the array has duplicates?

We'd need to decide: first occurrence or last? That's bisect_left vs bisect_right.

EASY | Binary Search | LeetCode #69 | 2 of 7 | 2/7

Sqrt(x)

QUESTION

Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

CONSTRAINTS

- For example, do not use pow(x, 0.5) in c++ or x ** 0.5 in python.

Example 1	
Input	x = 4
Output	2
Explanation	*The square root of 4 is 2, so we return 2.*

Example 2	
Input	x = 8
Output	2
Explanation	*The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.*

- $0 <= x <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return floor(sqrt(x)) without using built-in sqrt or ** 0.5. Right?

x can be 0 — sqrt(0)=0. x can be up to 2^31-1."

"x=4 -> 2. x=8 -> 2 (floor of 2.828). x=0 -> 0. x=1 -> 1."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
i = 0
while (i + 1) * (i + 1) <= x:
    i = i + 1
return i
```

TIME	SPACE
O(sqrt(x))	O(1)

WHY IT IS SLOW

Counts up one number at a time, about sqrt(x) steps (46,000 for 2^31). Binary search on the answer is O(log x), and Newton's method is faster still.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Binary search on the answer space [0, x].

Find the largest m such that m*m <= x.

When m*m < x, record m as a candidate and search right.

When m*m > x, search left. When m*m == x, return immediately.

Time: O(log x). Space: O(1)."

Key insight: the answer space [0, x] is monotone (m*m <= x increases with m) — binary search on the answer directly gives O(log x).

PHASE 4 CLEAN CODE

```
class _69_Sqrt:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x
        left, right = 1, x // 2   # sqrt(x) <= x//2 for x
>= 4
        result = 0
        while left <= right:
            mid = left + (right - left) // 2
            if mid * mid == x:
                return mid
            elif mid * mid < x:
                result = mid
                left = mid + 1
            else:
                right = mid - 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace mySqrt(8)."

Binary search on [0..8]. lo=0, hi=8. mid=4: 4^2=16>8 -> hi=3.

lo=0, hi=3. mid=1: 1<8 -> lo=2.

lo=2, hi=3. mid=2: 4<8 -> lo=3.

lo=3, hi=3. mid=3: 9>8 -> hi=2. lo=3>hi=2 -> return hi=2 ok (floor of 8~2.83).

mySqrt(4): mid=2->4==4->return 2 ok. mySqrt(1): hi=0+1=1->return 1 ok.

WHY binary search: finding x where x^2<=n. Monotonic -> binary search works.

WHY return hi not lo: when loop ends lo>hi. hi is the last valid floor(sqrt(n)). lo would overshoot.

mySqrt(0)=0: special case. mySqrt(2147395600)=46340 ok (boundary test).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log x)	O(1)

Using x//2 as the right bound instead of x halves the search space for large x —

worth mentioning to show you thought about the bounds.

mid*mid can overflow 32-bit integers in Java/C++ — in Python ints are arbitrary

precision so no issue, but worth flagging for the interviewer.

PHASE 6 COMPLEXITY & FOLLOW-UP

Given more time I'd ask: what about Newton's method?

x_{n+1} = (x_n + x/x_n) / 2 converges quadratically — faster in practice.

IN YOUR APP | Expose | Editorial | Community

EASY | Binary Search | LeetCode #278 | 3 of 7 | 3/7

First Bad Version

QUESTION

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API bool isBadVersion(version) which returns whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1

Input	`n = 5, bad = 4`
Output	`4`
Explanation	*call isBadVersion(3) -> false* *call isBadVersion(5) -> true* *call isBadVersion(4) -> true* *Then 4 is the first bad version.*

Example 2

Input	`n = 1, bad = 1`
Output	`1`

CONSTRAINTS

- 1 <= bad <= n <= 2^{31} - 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Versions 1..n. Once a version is bad, all subsequent are bad.

Find the first bad version while minimising API calls. Right?

`isBadVersion(k) is given — we don't implement it, just call it."`

"n=5, bad=4: isBadVersion(3)=F, isBadVersion(5)=T, isBadVersion(4)=T -> 4 ok

We want to minimise calls -> binary search, not linear scan."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for v = 1, 2, 3, ...:
    if isBadVersion(v):
        return v
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) calls	**O(1)**

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and an expensive API that is far too many. Versions are monotonic (good then bad), so binary search needs O(log n) calls.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search on first bad version — O(log n). Walk-through:"

lo=1, hi=n. While lo < hi:

`mid = (lo+hi)//2.`

If isBadVersion(mid): hi = mid. <- bad: first bad is at mid or before

Else: lo = mid+1. <- good: first bad is after mid

Return lo.

WHY hi=mid (not mid-1): mid itself could be the first bad version.

WHY lo<hi (not <=): when lo==hi we have the answer.

WHY minimize isBadVersion calls: O(log n) calls vs O(n) linear scan.

The first bad version is a monotone property: once true, always true.

Binary search on monotone properties always converges.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _278_FirstBadVersion:
    def firstBadVersion(self, n: int) -> int:
        left, right = 1, n
        while left <= right:
            mid = (left + right) // 2
            if isBadVersion(mid):
                right = mid - 1   # might be first bad; ke
ep searching left
            else:
                left = mid + 1    # definitely good; first
 bad is after mid
        return left
```

PHASE 5 TEST & DEBUG

"Let me trace firstBadVersion(5) assuming version 4 is first bad."

Binary search: lo=1, hi=5

mid=3: isBadVersion(3)=False -> lo=4

mid=4: isBadVersion(4)=True -> hi=4

lo==hi=4 -> return 4 ok

WHY binary search: sorted property — all good versions before first bad, all bad after.

isBadVersion is expensive (API call); binary search minimizes calls to O(log n).

PHASE 5 TEST & DEBUG

Edge: n=1, isBadVersion(1)=True -> return 1 immediately.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) calls	**O(1)**

The 'right = mid - 1' (not mid) is critical — if we set right=mid, when bad=1

the loop would never terminate. Always shrink the window.

Given more time I'd ask: what if the API is expensive (network call)?

Same algorithm — binary search already minimises calls. You'd want to batch

or cache results if the same version could be queried multiple times.

IN YOUR APP | Expose | Editorial | Community

EASY | Binary Search | LeetCode #367 | 4 of 7 | 4/7

Valid Perfect Square

QUESTION

Given a positive integer num, return true if num is a perfect square or false otherwise.

A perfect square is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as sqrt.

Example 1	
Input	`num = 16`
Output	`true`
Explanation	*We return true because 4 * 4 = 16 and 4 is an integer.*

Example 2	
Input	`num = 14`
Output	`false`
Explanation	*We return false because 3.742 * 3.742 = 14 and 3.742 is not an integer.*

CONSTRAINTS

- $1 <= num <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if num is a perfect square (square of a positive integer). No sqrt(). Right?"

"A few quick questions:

num >= 1 per constraints. Is 1 a perfect square? Yes — $1^2=1$."

"Let me walk: num=16 -> $4^2=16$ -> true ok. num=14 -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i from 1 while i * i <= num:
    if i * i == num:
        return True
return False
```

TIME	SPACE
O(sqrt(num))	O(1)

WHY IT IS SLOW

Tries every candidate root. Binary search on the root is O(log num); Newton's method is faster still.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the linear scan up to sqrt(n).

Binary search on the answer space [1, num//2+1]:

Find largest i where i*i <= num; check if i*i == num.

Time: O(log n). Space: O(1)."

Key insight: i*i is monotonically increasing — binary search finds the largest i with i*i <= num in O(log num), then a single equality check confirms a perfect square.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _367_ValidPerfectSquare:
    def isPerfectSquare(self, num):
        left, right = 1, num
        while left <= right:
            mid = (left + right) // 2
            if mid * mid == num:
                return True
            elif mid * mid < num:
                left = mid + 1
            else:
                right = mid - 1
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

num=16: l=1,r=16. mid=8->64>16->r=7. mid=4->16==16 -> True ok

num=14: l=1,r=14. mid=7->49>14->r=6. mid=3->9<14->l=4. mid=5->25>14->r=4.

mid=4->16>14->r=3. l>r -> False ok

num=1: mid=1->1==1 -> True ok

Mid*mid may be large but Python handles arbitrary-precision integers

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

Alternative: use mathematical property that perfect squares are sums of odd numbers (1+3+5+...).

Follow-up: Sqrt(x) (#69) — return the integer square root; same binary search.

Follow-up: Count Numbers with Unique Digits (#357) — different constraint, DP approach.

IN YOUR APP | Expose | Editorial | Community

EASY | Binary Search | LeetCode #374 | 5 of 7 | 5/7

Guess Number Higher or Lower

QUESTION

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n. You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API int guess(int num), which returns three possible results:

CONSTRAINTS

- -1: Your guess is higher than the number I picked (i.e. num > pick).
- 1: Your guess is lower than the number I picked (i.e. num < pick).
- 0: your guess is equal to the number I picked (i.e. num == pick).

Return the number that I picked.

Example 1	
Input	`n = 10, pick = 6`
Output	`6`

Example 2	
Input	`n = 1, pick = 1`
Output	`1`

Example 3	
Input	`n = 2, pick = 1`
Output	`1`

- $1 <= n <= 2^{31} - 1$
- 1 <= pick <= n

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Binary search with an API: guess(num) returns 0 (correct), -1 (too high),

or 1 (too low). Find the picked number. Right?

Range is 1..n, n up to 2^31-1."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for g in range(1,_n_+_1):
    if guess(g) == 0:
        return g
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

One guess per number. The hint says higher or lower, so binary search needs O(log n) guesses.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search on [1, n] — O(log n). Walk-through:"

lo=1, hi=n.

While lo < hi:

`mid = (lo+hi)//2.`

`result = guess(mid).`

If result == 0: return mid (guessed correctly).

If result == -1: hi=mid-1 (too high, guess lower).

If result == 1: lo=mid+1 (too low, guess higher).

Return lo.

WHY binary search: each guess eliminates half the range.

The answer is always in [lo, hi] after each step — binary search is monotone here.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _374_GuessNumber:
    def guessNumber(self, n: int) -> int:
        left, right = 1, n
        while left <= right:
            mid = left + (right - left) // 2   # avoids ov
erflow
            result = guess(mid)
            if result == 0:
                return mid
            elif result == -1:
                right = mid - 1
            else:
                left = mid + 1
        return left
```

PHASE 5 TEST & DEBUG

"Let me trace guessNumber(10, pick=6)."

Binary search: lo=1, hi=10

mid=5: guess(5)=1 (too low) -> lo=6

mid=8: guess(8)=-1 (too high) -> hi=7

mid=6: guess(6)=0 (correct) -> return 6 ok

WHY binary search: guess API tells direction; O(log n) guesses needed.

PHASE 5 TEST & DEBUG

`guess() returns -1 (too high), 0 (correct), 1 (too low).`

Edge: pick=1 -> mid=5 too high->hi=4->mid=2 too high->hi=1->mid=1 correct.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

Using left + (right-left)//2 instead of (left+right)//2 avoids integer overflow

in languages with fixed-width integers — worth mentioning even in Python.

Given more time I'd ask: what if guess() had a cost per call?

Binary search already minimises calls to O(log n) — optimal.

IN YOUR APP | Expose | Editorial | Community

EASY | Binary Search | LeetCode #704 | 6 of 7 | 6/7

Binary Search

QUESTION

Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.

You must write an algorithm with O(log n) runtime complexity.

Example 1	
Input	`nums = [-1,0,3,5,9,12], target = 9`
Output	`4`
Explanation	*9 exists in nums and its index is 4*

Example 2	
Input	`nums = [-1,0,3,5,9,12], target = 2`
Output	`-1`
Explanation	*2 does not exist in nums so return -1*

CONSTRAINTS

- $1 <= nums.length <= 10^4$
- $10^4 < nums[i], target < 10^4$
- All the integers in nums are unique.
- nums is sorted in ascending order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Classic binary search: sorted array of distinct integers, find target index.

Return -1 if not found. Must be O(log n). Right?"

"[-1,0,3,5,9,12], target=9 -> 4 ok

[-1,0,3,5,9,12], target=2 -> -1 ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    if nums[i] == target:
        return i
return -1
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Ignores that the array is sorted. Binary search halves the range each step: O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Classic binary search — O(log n). Walk-through:"

lo=0, hi=n-1.

While lo <= hi:

`mid = lo + (hi-lo)//2 <- avoids integer overflow vs (lo+hi)//2.`

If nums[mid] == target: return mid.

If nums[mid] < target: lo=mid+1 (target is in right half).

Else: hi=mid-1 (target is in left half).

Return -1 (not found).

WHY lo + (hi-lo)//2: mathematically identical to (lo+hi)//2 but avoids overflow in

languages with fixed int size. Good habit to mention in interviews.

Invariant: target is always in [lo, hi] if it exists.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _704_BinarySearch:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] > target:
                right = mid - 1
            else:
                left = mid + 1
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace search([-1,0,3,5,9,12], target=9)."

Binary search: lo=0, hi=5

mid=2: nums[2]=3 < 9 -> lo=3

mid=4: nums[4]=9 == 9 -> return 4 ok

Trace target=2: lo=0,hi=5; mid=2:3>2->hi=1; mid=0:-1<2->lo=1; mid=1:0<2->lo=2; lo>hi -> return -1 ok

WHY standard binary search: sorted array -> O(log n) instead of O(n) linear scan.

Loop condition: lo<=hi; mid=(lo+hi)//2; adjust lo or hi based on comparison.

Invariant: target always in [lo,hi] if it exists; lo>hi means not found -> -1.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

This is the template all other binary search problems build on.

Key invariant: loop condition is left <= right (not <) so single-element

arrays are handled — the loop runs once and either returns or terminates cleanly.

Given more time I'd ask: what if there are duplicates and we want the first/last

occurrence? That's leftmost/rightmost binary search — adjust which boundary

updates when nums[mid]==target.

IN YOUR APP | Expose | Editorial | Community

EASY | Binary Search | LeetCode #744 | 7 of 7 | 7/7

Find Smallest Letter Greater Than Target

QUESTION

You are given an array of characters letters that is sorted in non-decreasing order, and a character target. There are at least two different characters in letters.

Return the smallest character in letters that is lexicographically greater than target. If such a character does not exist, return the first character in letters.

Example 1

Input	`letters = ["c","f","j"], target = "a"`
Output	`"c"`
Explanation	*The smallest character that is lexicographically greater than 'a' in letters is 'c'.*

Example 2

Input	`letters = ["c","f","j"], target = "c"`
Output	`"f"`
Explanation	*The smallest character that is lexicographically greater than 'c' in letters is 'f'.*

Example 3

Input	`letters = ["x","x","y","y"], target = "z"`
Output	`"x"`
Explanation	*There are no characters in letters that is lexicographically greater than 'z' so we return letters[0].*

CONSTRAINTS

- 2 <= letters.length <= 10^4
- letters[i] is a lowercase English letter.
- letters is sorted in non-decreasing order.
- letters contains at least two different characters.
- target is a lowercase English letter.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the smallest character in letters that is strictly greater than target.

If none, return letters[0] (wrap around). Right?"

"A few quick questions:

PHASE 1 CLARIFY

letters is sorted and has at least 2 unique characters? Yes.

Wrap around meaning the list is circular? Yes."

"Let me walk: letters=['c','f','j'], target='a' -> 'c' ok. target='d' -> 'f' ok. target='j' -> 'c' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for letter in order:
    if it is greater than target:
        return it
return letters[0]
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Linear scan of a sorted array. Binary search for the first letter greater than target (wrapping to letters[0]) is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the linear scan — letters is sorted so we can binary search.

Find the leftmost index where letters[i] > target using bisect_right.

If index == len(letters), wrap to letters[0].

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _744_FindSmallestLetterGT:
    def nextGreatestLetter(self, letters, target):
        import bisect
        idx = bisect.bisect_right(letters, target)
        return letters[idx % len(letters)]
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

letters=['c','f','j'], target='a': bisect_right(['c','f','j'],'a')=0. letters[0]='c' ok

target='d': bisect_right=1. letters[1]='f' ok

target='j': bisect_right=3. 3%3=0. letters[0]='c' ok (wrap around)

target='c': bisect_right=1. letters[1]='f' ok (strictly greater, not equal)

PHASE 5 TEST & DEBUG

Bisect_right gives first index where letters[i] > target — strict inequality

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

bisect_right handles duplicates correctly — skips equal elements.

Follow-up: Find Smallest Letter Greater Than Target with k wraps — mod arithmetic.

Follow-up: if duplicates exist in letters, bisect_right still works correctly.

IN YOUR APP | Expose | Editorial | Community